

PROYECTO IS MANUAL DE ESTÁNDARES Y ARQUITECTURA

1. GENERALIDADES Y FLUJO DE TRABAJO

1.1 Configuración de Lenguajes y Tipado

- **Uso obligatorio de TypeScript:** Todo el código del proyecto debe desarrollarse estrictamente en TypeScript.
- **Excepción de JavaScript:** Sólo se aceptarán Pull Requests (PR) con archivos JavaScript en casos excepcionales por incompatibilidad de bibliotecas o ausencia de tipos oficiales. Si existen definiciones de tipos (`@types`), el PR con JavaScript será rechazado.

1.2 Estrategia de Ramas en Git

Se implementa una nomenclatura estandarizada para ramas:

- **Estructura:** `<branch>/<group-num>-<description-of-branch>` (Ejemplo: `feature/group-1-login`).
- **Tipos de rama permitidos:** `feature`, `fix`, `hotfix` (uso prioritario de líderes o DevOps), `refactor`, `docs`, `test` y `epic`.
- **Exclusividad DevOps:** Las ramas del tipo `chore` son de uso exclusivo del equipo DevOps.

1.3 Estándar de Commits

Se adoptan las convenciones de *Conventional Commits* e influencias del estándar de Angular:

- **Tipos autorizados:** `docs`, `feat`, `fix`, `perf`, `refactor`, `test`, `build`.
- **Estructura:** `feat: message`, `feat(frontend): message`
- **Restricción estricta:** Se rechazará cualquier commit que contenga cambios directos sobre integración continua (CI).

1.4 Gitflow Adaptado y Ambientes

1. Ramas Principales de Despliegue:

- `main` (Producción): Contiene la versión estable oficial. Se actualiza exclusivamente al finalizar cada Sprint.
- `develop` (Desarrollo Continuo): Entorno de pruebas principal. Recibe actualizaciones constantes (cada 2 a 5 días) integrando el trabajo validado por el equipo DevOps.

2. Gestión de Épicas (Gitflow Interno):

- **Ramas `epic/*`:** Cada grupo posee su propia rama base (Ejemplo: `epic/grupo1-red-de-mentores`).
- **Ramas de Tarea:** Cada miembro de Equipo se derivan desde la rama `epic` correspondiente mediante ramas transitorias (`feature/*`, `fix/*`, `hotfix/*`).

1.5 Reglas de Integración y Pull Requests (PR)

- **Integración hacia epic/* (Líderes de Equipo):** Los desarrolladores envían PRs a su rama epic. El líder de equipo aprueba o rechaza evaluando estándares de código, arquitectura y separación de responsabilidades.
- **Integración hacia develop (DevOps):** Cada 2 a 5 días, el líder de equipo envía un PR desde epic/* hacia develop. Solo el equipo DevOps está autorizado para aprobar este pase a fin de prevenir inestabilidad global.
- **Horarios de revisión del equipo DevOps:** No se revisará más allá del horario 9am-10pm.

1.6 Manejo de Dependencias y Bloqueos entre Equipos

- **Desbloqueo temporal:** Si un equipo depende de una implementación no disponible de otro grupo, Se establece un límite **2 días**. Cumplido el plazo, el equipo perjudicado queda autorizado a desarrollar utilizando IA, se hará más flexible la revisión pero en la PR poner (hasta comentar) que archivos tuvieron que implementar sobre los equipos y no se aprobara PR al equipo que no implemento hasta que arregle estos archivos “vibe codeados”, también se debe hacer algún informe sobre estos.
- **Protocolo de Integración Cruzada:** Queda estrictamente prohibido realizar *merge* directo entre dos ramas epic/* distintas. El flujo obligatorio es:
 1. **Publicación:** El equipo proveedor hace PR desde su rama epic hacia develop.
 2. **Aprobación:** DevOps aprueba e integra los cambios en develop.
 3. **Consumo:** El equipo cliente hace *pull/merge* de develop actualizado hacia su propia rama epic.

1.7 Gestión de Proyecto y Control de Calidad (CI/CD)

- **Gestión de Tareas:** Se utilizará GitHub Projects dentro de una organización pública. Cada equipo dispondrá de un tablero KANBAN para sus *issues*, además de un KANBAN general de Historias de Usuario visible para todo el proyecto.
- **Automatización en Frontend y Backend:**
 - **Cobertura de Pruebas (Coverage):** Mínimo requerido del **80%**.
 - **Linter:** Si el código no cumple las reglas de *linting*, DevOps rechazará el PR hacia develop.
 - Los equipos podrán utilizar IA para el desarrollo de tests unitarios para cumplir el coverage mínimo.

1.8 Contrato de respuesta estandarizada con backend hacia clientes

- Las respuesta JSON tienen que tener los siguientes campos:
 - statusCode
 - data
 - offset (opcional según dependa)
 - page (opcional según dependa)
 - detail

- ok

1.9 Estándar de los endpoints

- Se tendrá que seguir la convención Rest.

2. ESTÁNDARES GLOBALES DE CÓDIGO Y NOMENCLATURA

2.1 Idioma y Contexto

- **Código Técnico en Inglés:** Todo nombre de variable, función, clase, interfaz, archivo, actions, carpeta y entidad de base de datos se escribirá estrictamente en inglés.
- **Texto y Documentación en Español:** Se usará español obligatoriamente para:
 1. Interfaz de Usuario (UI): Mensajes, etiquetas, botones, modales y alertas.
 2. Comentarios de Código: Explicaciones técnicas y documentación interna de lógica compleja.
 3. Ni un emoji en ningún lado, ni en la UI ni comentarios.

2.2 Convenciones de Formato

- **camelCase (Variables, Funciones y Métodos):**
 - Funciones: Iniciar con un verbo de acción (`getUserData`, `calculateTotal`, `handleFormSubmit`).
 - Booleanos: Iniciar con prefijos de condición (`isOpen`, `hasError`, `canSubmit`).
- **PascalCase (Componentes, Clases, Tipos e Interfaces):**
 - Nombres representativos (`UserProfileCard`, `AuthService`).
 - Sufijos descriptivos obligatorios para contratos de datos (`Props`, `Response`, `Payload`, `Exception`).
- **kebab-case (Archivos y Directorios):** Todo nombre de archivo o carpeta en el repositorio irá en minúsculas separado por guiones (`user-profile.tsx`, `auth-service.ts`) para garantizar compatibilidad entre sistemas operativos.
- **UPPER_SNAKE_CASE (Constantes Globales):** Para valores inmutables y configuraciones globales (`MAX_UPLOAD_SIZE`, `API_BASE_URL`).

2.3 Manejo de Rutas Dinámicas

- Formato con corchetes en minúsculas: `[id]`, `[slug]`, `[token]`. Se prioriza una sola palabra clara dentro del parámetro.

2.4 Patrón de Acceso "Barril" (`index.ts`)

- Los archivos `index.ts` o `index.tsx` actúan **únicamente** como puentes de re-exportación.

- **Prohibición:** Queda prohibido programar lógica de negocio, algoritmos o renderizado UI dentro de un archivo `index`.

3. ARQUITECTURA Y ESTÁNDARES DEL FRONTEND

3.1 Stack Tecnológico

- **Core:** Next.js 16 (App Router), React 19.2.
- **Estilos y UI:** TailwindCSS, shadcn/ui, Solo se permitirá Lucide Icons
- **Gestión de estado servidor:** TanStack Query (React Query) v5
- **Estado de sesión/UI global:** React Context (`AuthContext`) para sesión
- **Persistencia local:** `idb` (wrapper sobre IndexedDB) para datos de negocio y cola de sincronización; `localStorage` solo para preferencias livianas (tema, idioma).
- **Arquitectura:** Modular por Dominio

1. Colores:

2. Estilos:

3.2 Responsabilidades por Capa y Módulo

1. Capa de Enrutamiento (`app/`)

Responsabilidad: definir rutas, layouts y estados de carga/error. **No** ejecuta obtención de datos para SEO

Regla estricta: un `page.tsx` será solo un Server Component delgado que solo resuelve `params/searchParams` (que en Next 16 son `Promise`, hay que `await`-earlos) y renderiza la View importada del módulo, nunca contiene JSX de negocio.

2. Capa Modular por Funcionalidad (`modules/`)

Sin cambios de filosofía: se organiza por dominio de negocio permanente (`modules/auth`, `modules/billing`, `modules/enrollment`, `modules/scanner`). Estructura interna, con una adición:

- `views/`: (**nuevo**) el componente de pantalla completa que antes vivía suelto (ej. `DashboardView`) ahora tiene casa propia, separado de los componentes reutilizables del módulo.
- `components/`: piezas visuales internas del módulo.
- `hooks/`: `useQuery/useMutation` de TanStack Query, específicos del dominio.
- `services/`: llamadas HTTP del dominio.

- `types/`, `index.ts`: igual que antes.

Toda View y todo hook que use estado de React, TanStack Query o APIs del navegador (cámara, IndexedDB) debe declararse `'use client'` explícitamente.

3. Capa de Componentes Compartidos (`shared/components/`)

Sin cambios: `ui/` (primitivos shadcn) y `layout/` (Navbar, Sidebar, Shell). Se agregan `AuthGate` y `RoleGate` como componentes de layout puros que reciben el rol/estado de sesión por props o contexto y deciden qué renderizar, nunca hacen fetch directo.

4. Capa de Servicios Core (`shared/services/`)

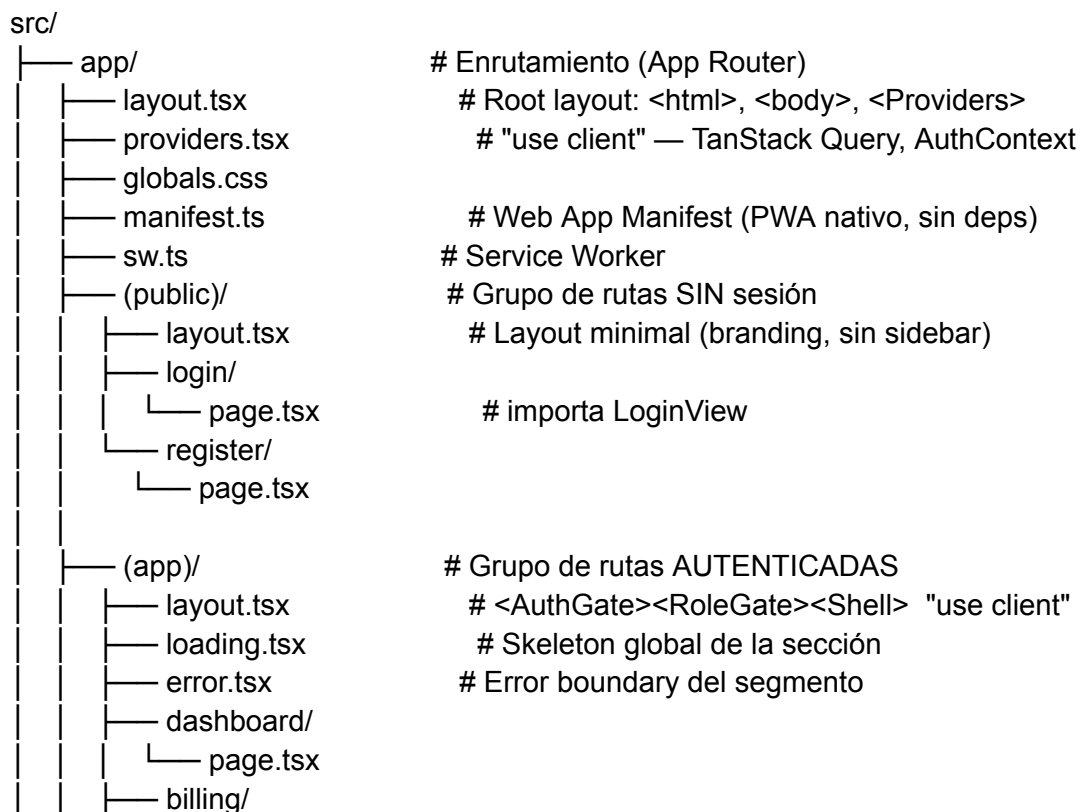
Instancia única de Axios (`api-client.ts`) con interceptores para 401/refresh. Sin cookie de sesión puesta por un backend propio, así que el token (y el refresh token) vive en IndexedDB. Prohibición estricta: nadie importa `axios` o usa `fetch` directo fuera de esta instancia.

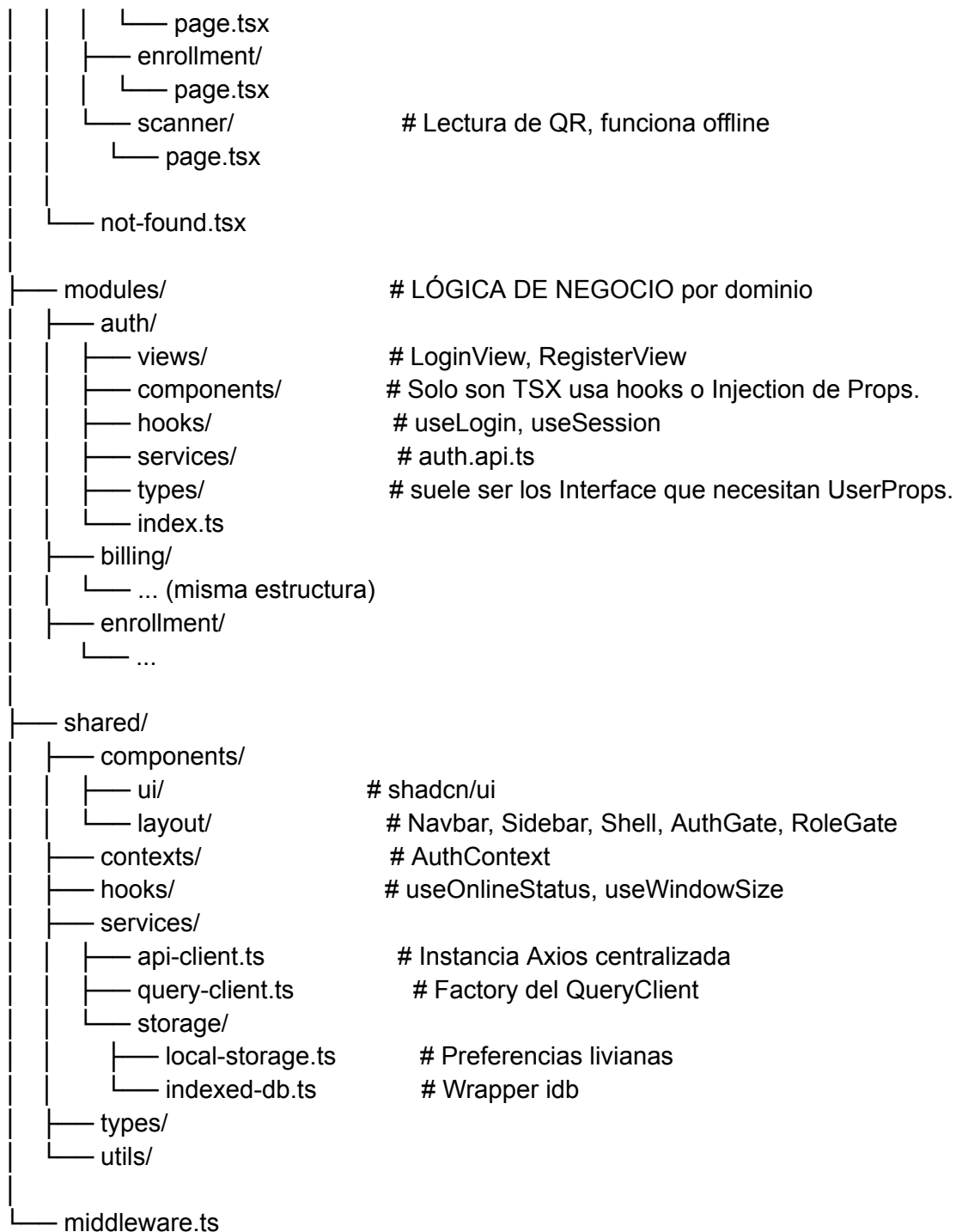
5. Capa de Persistencia (`shared/services/storage/`) centraliza el acceso a `localStorage` (preferencias) e IndexedDB (`idb`) para datos de negocio y para el persistir de TanStack Query. Ningún módulo abre IndexedDB directamente, todos pasan por este wrapper.

6. Capa de Estilos

solo utilidades Tailwind, cero CSS por componente, solo existirá `globals.css`

3.3 Estructura de Directorios





3.4 Reglas de Arquitectura y Buenas Prácticas

1. **Manejo de errores estandarizado:** los errores de infraestructura (401/403/500) se interceptan en `api-client.ts`; los de negocio (400/404/422) se propagan al módulo y se muestran como toasts en español vía `onError` de TanStack Query.
2. **Encapsulamiento estricto vía `index.ts`:** cada módulo exporta explícitamente sus Views, hooks y tipos públicos. Todo lo que no se exporte es privado del módulo.

3. **Cero acoplamiento cruzado entre dominios:** ningún módulo importa desde las subcarpetas internas de otro.
4. **Frontera Server/Client explícita** (*nueva*): `page.tsx` y `layout.tsx` en `app/` se mantienen como Server Components delgados. Todo lo que dependa de hooks de React, TanStack Query, `window`, cámara o IndexedDB va en un archivo `'use client'` dentro de `modules/*/views` o `modules/*/components`.
5. **Alcance del Service Worker:** Swisw solo cachea el app shell y assets estáticos. Nunca contiene lógica de negocio, eso vive en `shared/services/storage` y en los hooks de cada módulo.

4. ARQUITECTURA Y ESTÁNDARES DEL BACKEND

4.1 Stack Tecnológico y Patrón Arquitectónico

- **Tecnologías:** NestJS, Prisma ORM, PostgreSQL.
- **Arquitectura:** Monolítica Modular.

4.2 Responsabilidades por Capa y Módulo

Controladores (`Controllers`)

- Reciben la petición HTTP entrante previamente validada por los esquemas Zod.
- Invocan el método correspondiente del servicio del módulo.
- **Prohibición:** No contienen lógica de negocio bajo ninguna circunstancia. Delegación de formato de respuesta final a interceptores globales.

Servicios (`Services`)

- Contienen el **100% de la lógica de negocio** del módulo.
- **Exclusividad:** Son los únicos componentes autorizados para inyectar Prisma Service y realizar persistencia en base de datos.
- Ejecutan cálculos, transformaciones complejas y lanzan excepciones de negocio personalizadas.

Filtros de Excepciones (`Exception Filters`)

- Capturan las excepciones personalizadas lanzadas por los servicios.
- Mapean y formatean el error en una estructura JSON estandarizada con código de estado HTTP y mensaje explicativo.
- Solo una excepción por archivo.

Interceptores (`Interceptors`)

- Capturan las respuestas exitosas devueltas por los controladores.
- Envuelven los datos automáticamente en el formato unificado de la aplicación:

Esquemas y Validación (`Schemas - Zod`)

- Definen la estructura y tipos exactos de los datos de entrada para operaciones de creación y actualización. Sustituyen las validaciones por anotaciones tradicionales.

4.3 Reglas de Interacción entre Módulos

- **Prohibición de Acceso Directo:** Un módulo no debe consultar directamente las tablas o esquemas de base de datos de otro dominio.
- **Comunicación Exclusiva:** Para acceder a datos de otro dominio, se debe inyectar el **Servicio** del módulo requerido dentro del servicio del módulo solicitante.
- Cada módulo encapsula sus propias excepciones de negocio.

4.4 Nomenclatura y Estándares de Archivos

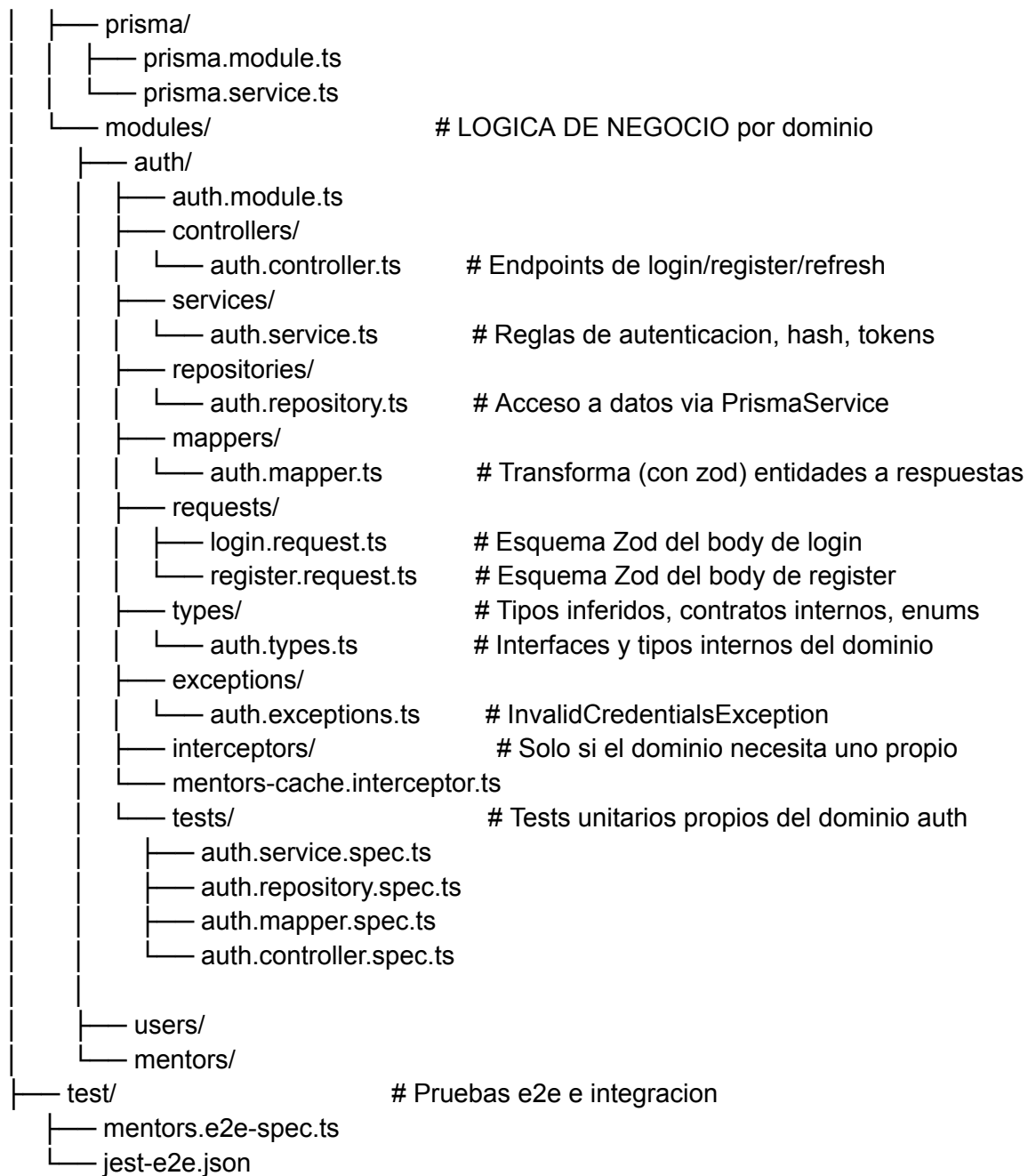
- Directores en minúsculas **kebab-case**.
- Nombres de archivos según estándar NestJS:
 - Controladores: **module-name.controller.ts**
 - Servicios: **module-name.service.ts**
 - Módulos: **module-name.module.ts**
 - Esquemas: **module-name.schema.ts**
 - Excepciones: **module-name.exceptions.ts**
- Clases de excepción personalizadas finalizan obligatoriamente con el sufijo **Exception** (Ejemplo: **UserNotFoundException**).

4.5 Estructura de archivos ejemplo.

```

backend/
├── prisma/
│   ├── schema.prisma
│   └── migrations/
├── src/
│   ├── main.ts
│   ├── app.module.ts           # Modulo raiz: registra filtros, modulos de dominio y config
│   ├── config/                # Configuracion y validacion de entorno
│   │   ├── configuration.ts    # Carga variables tipadas desde .env
│   │   └── env.validation.ts   # Esquema Zod para validar el .env al arrancar
│   ├── common/                # Infraestructura transversal (no es dominio)
│   │   ├── exceptions/
│   │   │   └── domain.exception.ts    # Clase base abstracta: DomainException
│   │   ├── filters/
│   │   │   ├── http-exception.filter.ts    # Atrapa HttpException nativas de Nest
│   │   │   ├── prisma-exception.filter.ts
│   │   │   └── domain-exception.filter.ts    # Atrapa DomainException y sus hijas
│   │   ├── interceptors/
│   │   │   └── response.interceptor.ts    # Envuelve respuestas exitosas en el formato
│   │   ├── pipes
│   │   ├── decorators/
│   │   ├── guards/            # Guards de autenticacion / autorizacion
│   │   ├── utils/             # Helpers puros fechas, strings, paginacion
│   │   ├── tests/             # Tests de la infraestructura transversal
│   │   └── response.interceptor.spec.ts

```



5. ESTÁNDARES Y ARQUITECTURA DE BASE DE DATOS (PRISMA & POSTGRESQL)

Se recomienda que la base de datos sea diseñada únicamente por personas que hayan aprobado TBD con el ingeniero Flores.

5.1 Gobernanza y Gestión de Esquema

- **Control de Modificaciones:** Diseñado bajo los lineamientos base del proyecto, el equipo de **DevOps** es el único autorizado para modificar y aplicar cambios sobre el archivo **schema.prisma** y ejecutar migraciones en producción.

5.2 La Regla del "Doble Estándar" (Mapeo de Prisma)

PostgreSQL (Base de Datos) y TypeScript (Código Fuente) poseen convenciones de nomenclatura distintas. Es **obligatorio** el uso de las directivas `@map` y `@@map` en Prisma para garantizar la compatibilidad.

Modelos y Tablas

- **Código Fuente (Prisma Model):** Singular y `PascalCase` (Ejemplo: `UserProfile`).
- **Base de Datos (PostgreSQL Table):** Plural y `snake_case` (Ejemplo: `user_profiles`).

Campos, Columnas y Auditoría

- **Código Fuente (Prisma Field):** `camelCase` (Ejemplo: `firstName`, `createdAt`).
- **Base de Datos (PostgreSQL Column):** `snake_case` (Ejemplo: `first_name`, `created_at`).
- **Auditoría Obligatoria:** Toda tabla transaccional debe incluir campos de auditoría temporal con decoradores nativos de Prisma.

Claves Primarias (`id`)

- Toda tabla debe definir su clave primaria nombrada **únicamente** como `id`.
- **Prohibición:** Queda prohibido incluir el nombre de la tabla en su propia clave primaria (Ejemplo: `userId` dentro de la tabla `User`).

Claves Foráneas y Relaciones

- **Relación Virtual (TypeScript):** En singular para relaciones 1:1 o N:1 (Ejemplo: `teacher`), en plural para relaciones 1:N (Ejemplo: `posts`).
- **Columna Física (PostgreSQL):** Nombre en `snake_case` finalizado obligatoriamente en `_id`.

Enumeraciones (Enums)

- **Código Fuente (TypeScript):** Nombre en singular y `PascalCase` (`UserRole`), valores en `UPPER_SNAKE_CASE` (`ADMIN`).
- **Base de Datos (PostgreSQL):** Mapeo a plural y `snake_case` para el tipo, y valores en minúscula.

Idioma del Esquema

- Tablas, columnas, relaciones y enumeraciones se declaran estrictamente en **inglés**.
- Los datos de prueba (`seeds`) pueden redactarse en **español**.

6. ESTÁNDARES PARA LA CREACIÓN DE ISSUES

El objetivo de estas plantillas es asegurar que cualquier integrante del equipo tenga todo el contexto técnico, el alcance y las condiciones de entrega antes de empezar a programar.

6.1 Estructura del Formulario

Cada vez que se registre una nueva tarea en GitHub (Frontend, Backend o General), el formulario solicitará los siguientes campos en orden:

Campo	Carácter	Qué se debe ingresar	Ejemplo
1. Epic e Historia de Usuario	Obligatorio	La Epic y la Historia de Usuario a la que pertenece la tarea. Debe iniciar siempre con el formato Epic #N	Epic #8 - Historia de Usuario 01 (Inicio de sesión)
2. Tipo de tarea	Obligatorio	Selección de la categoría del cambio dentro del menú desplegable.	Feature, fix, refactor, test, docs o hotfix
3. Prioridad	Obligatorio	Nivel de urgencia acordado para el tablero de trabajo.	Alta, Media o Baja
4. Estimación en horas	Obligatorio	Tiempo estimado de desarrollo expresado exclusivamente en horas.	4 horas
5. Descripción técnica	Obligatorio	Explicación detallada de lo que se va a construir o corregir (componentes, rutas, endpoints, lógica o base de datos).	Crear el servicio de autenticación y conectar el formulario...
6. Criterios de aceptación	Obligatorio	Lista de condiciones verificables (- []) que deben cumplirse para dar la tarea por terminada.	- [] Valida credenciales - [] Redirige al panel
7. Dependencias	Opcional	Indica si esta tarea necesita que otra Historia de Usuario (HU) esté finalizada antes de poder completarse.	Depende de la HU #15

6.2. Reglas importantes

- Formato (**Epic #N**): Para mantener el orden en el tablero, toda referencia debe comenzar con la palabra **Epic** seguida del símbolo **#** y el número correspondiente (por ejemplo: **Epic #3 - ...**).
- Está prohibido crear issues fuera del tablero del equipo, cualquier issue suelta, (fuera del tablero de su proyecto), será eliminada por el equipo de DevOps.
- Está prohibido crear issues en blanco, siempre se usa cualquiera de las plantillas dependiendo de a donde pertenece: fullstack, backend y frontend..

6. ANEXOS

Para algunos conceptos de la metodología de trabajo como convención de commits se recomienda las lecturas de:

- <https://github.com/angular/angular/blob/main/contributing-docs/commit-message-guidelines.md>
- <https://www.conventionalcommits.org/en/v1.0.0/>

Para informacion sobre APIREST

- <https://www.redhat.com/es/topics/api/what-is-a-rest-api>
- <https://aws.amazon.com/es/what-is/restful-api/>